To centrally implement a session-out dialog for 401 errors in Next.js, use an **Axios Interceptor** on the client-side to detect the status code, clear local auth tokens, and trigger a modal or redirect. For Server Actions, wrap actions in a handler that catches 401 errors and uses `redirect()`. GitHub +4

1. Client-Side Implementation (Axios Interceptors)

Create an API instance and add a response interceptor to handle 401 errors globally.

javascript

```javascript
// lib/api.js (or similar)
import axios from 'axios';
// Assume you have a mechanism to open a modal, e.g., a global store
import { useAuthStore } from '@/store/auth';

const apiClient = axios.create({
  baseURL: '/api',
});

apiClient.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response && error.response.status === 401) {
      // 1. Clear Local Storage/Cookies
      localStorage.removeItem('token');

      // 2. Trigger Session Out Dialog (via state management like Zustand/Redux)
      useAuthStore.getState().setSessionExpired(true);

      // 3. Or simply redirect to login
      // window.location.href = '/login?message=session_expired';
    }
    return Promise.reject(error);
  }
);

export default apiClient;
```

2. Server-Side Implementation (Server Actions/Fetch)

When using Server Actions, check for 401 responses and use `redirect` from `next/navigation`.

typescript

```typescript
// app/actions.ts
'use server'
import { redirect } from 'next/navigation';

export async function secureAction() {
  const response = await fetch('https://api.example.com/protected', {
    headers: { Authorization: 'Bearer ...' }
  });

  if (response.status === 401) {
    // Session expired
    redirect('/login?sessionExpired=true');
  }

  return response.json();
}
```

3. Displaying the Dialog (UI Component)

Use a global component (e.g., in `layout.js`) to watch for the `sessionExpired` state. Level Up Coding +1

jsx

```jsx
// components/SessionTimeoutModal.jsx
'use client'
import { useAuthStore } from '@/store/auth';
import { useRouter } from 'next/navigation';

export default function SessionTimeoutModal() {
  const { sessionExpired, setSessionExpired } = useAuthStore();
  const router = useRouter();

  if (!sessionExpired) return null;

  return (
    <div className="modal">
      <h2>Session Expired</h2>
      <p>Your session has timed out. Please log in again.</p>
      <button onClick={() => {
        setSessionExpired(false);
        router.push('/login');
```

```
      }}>
        Log In
      </button>
    </div>
  );
}
```

Key Considerations

- **Authentication Libraries:** If using NextAuth.js (Auth.js), the `signOut` function can be used within the interceptor to clear cookies.
- **Global State:** Zustand or React Context works well for triggering the dialog across the application.
- **Redirect vs. Modal:** A redirect to `/login` is often cleaner than a modal, as it forces a clean state. GitHub